

## Homework Week 5 [40 points + 10 extra points]

**Due: February 18, 2026 11:59pm PT.**

Upload a zip archive of your project on <https://brightspace.usc.edu>.

Your project must consist of a folder `hw5_<first_name>_<last_name>/` and python files (python modules) and data file(s) inside:

- `hw5_<first_name>_<last_name>/tuple_and_file_utils.py` # place your functions from tasks 1-4 here
- `hw5_<first_name>_<last_name>/grades_summary.py` # implement command line interface for task 4 here
- `hw5_<first_name>_<last_name>/tests.py` # place your tests here, see example in HW4
- `hw5_<first_name>_<last_name>/students.csv`

### Task 1. Tuple Transformation. [10 points]

Given a tuple `original_tuple` containing integers, write a Python function `transform_tuple(original_tuple)` that:

- Finds the sum of all even numbers.
- Finds the sum of all odd numbers.
- Returns a new tuple with the first element being the sum of all even numbers and the second the sum of all odd numbers.

Function signature:

- **`def transform_tuple(original_tuple) -> (int, int):`**

Arguments:

- **`original_tuple`**: tuple of int values (e.g. (234, 324, 5, 53,75) ).

Returns a tuple of two int values.

### Task 2. Unique letters. [10 points]

Write a Python function `unique_letters(string1, string2)` that takes two strings and returns a tuple where:

1. The first set contains characters unique to `string1`.
2. The second set contains characters unique to `string2`.
3. The third set contains characters present in both strings

Function signature:

- **`def unique_letters(string1:str, string2:str) -> (set,set, set):`**

# DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

---

Returns a tuple of three sets.

## Task 3. Grade summary function. [10 points + 10 extra points]

Implement a Python function **grade\_summary(student\_grades)** for a dictionary **student\_grades** where each key is a student's name and the value is a list of their grades. The function should return a new dictionary with the student's name as keys, and each value being another dictionary with two keys: **'average'** for the student's average grade and **'highest'** for their highest grade. Round off **'average'** to 2 decimal places..

Function signature:

- **def grade\_summary(student\_grades:dict) -> dict:**

Returns: dictionary.

Example:

Input: {"Alice": [88, 92, 100], "Bob": [75, 78, 80]} # number of grades can be different for each student

Output: {"Alice": {"average": 93.33, "highest": 100}, "Bob": {"average": 77.67, "highest": 80}}

Extra credit opportunity: You know that order of elements in the dictionary is undetermined by default (order of keys), but if you figure out how to sort elements in the output dictionary you can get 10 points. It will be tested as follows: `print(grade_summary(student_grades))`, where **student\_grades** dict is from **students.csv** and if student names are in alphabetical order in the print output order you get +10 points. You can assume that we are using python 3.12 (or higher) version.

## Task 4. Grade summary program. [10 points]

**(!) Read about `argparse` package in your lab\_week5.pdf. Friday's lab session will be reviewing this.**

Create a small program that manages student grades for a class. The program should read student data from a file, process the data by calculating grade summary (use function from task 3), and then write the results to another file.

Inputs: Input file is in CSV format (see example **students.csv**) will look like this:

```
student_id,student_name,grade1,grade2,grade3
1,John Doe,85,90,78
```

# DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

---

```
2,Jane Smith,92,,95
3,Bob Johnson,70,75,80
4,Alice Brown,88,,85
5,Charlie Davis,60,65,70
```

Number of grades can be different for each student (sometimes you can see a missing grade: Jane Smith,92,,95). Maximum number of grades is determined in the header of the CSV file (in the example above it is 3 grades, in *students.csv* it is 15 grades).

Outputs: output file should be in csv format as well, it should look like this

```
student_id,student_name,average_grade, highest
1,John Doe,84.33,90
2,Jane Smith,91.67,93
3,Bob Johnson,75.00, 80
```

Notice that the input and output file have student id but your function from task 3 doesn't use that. This is intentional, you have to do some input data pre processing, so that your function could work. You are not allowed to change interface or input/output format in function from task 3.

Your program should run from a terminal as follows:

***python grades\_summary.py students.csv summaries.csv***

Use the template below for *grades\_summary.py*:

```
import argparse
from tuple_and_file_utils import read_students, grade_summary, write_averages

def main():
    # Set up command line argument parsing, do lab assignment to practice using argparse
    library
    parser = argparse.ArgumentParser(description='Process student grades.')
    parser.add_argument('input_file', type=str, help='Input file containing student
grades')
    parser.add_argument('output_file', type=str, help='Output file to write average
grades')

    args = parser.parse_args()

    # Read students from the input file
    students = read_students(args.input_file)
    ... # some input processing if needed
```

# DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

---

```
# Calculate averages
summary = grade_summary(students)

# Write averages to the output file
write_averages(args.output_file, summary)

if __name__ == "__main__":
    main()
```

Also implement ***read\_students()*** and ***write\_averages()*** in ***tuple\_and\_file\_utils.py***

```
# implement reading student's grades from a file
def read_students(file_name: str) -> dict:
    ...

# implement writing student's grade summaries to a file
def write_averages(file_name: str, averages: dict):
    ...
```